

CSE 4304-Data Structures Lab. Winter 2024-25

Date: 18 February 2026

Target Group: All groups

Topic: DSU, Graph II.

Instructions:

- Task naming format: fullID_T01L01_2A.c/cpp
- If you find any issues in the problem description/test cases, comment in the Google Classroom.
- If you find any tricky test cases that I didn't include but that others might forget to handle, please comment! I'll be happy to add them.
- Use appropriate comments in your code. This will help you recall the solution more easily in the future.
- The obtained marks will vary based on the efficiency of the solution.
- Do not use <bits/stdc++.h> library.
- Modified sections will be marked with **BLUE** color.
- You are allowed to use the STL stack unless it's specifically mentioned to use manual functions.

Group	Tasks
1A/1B/2A/2B	1 2 3 4
Assignment	

Task 1: Connect the cities!

There are n cities and initially no roads between them. However, every day a new road will be constructed, and there will be a total of m roads.

A component is a group of cities where there is a route between any two cities using the roads. After each day, your task is to find the number of components and the size of the largest component.

Input:

- The first input line has two integers n and m : the number of cities and roads.
- The cities are numbered $1, 2, \dots, n$.
- Then, there are m lines describing the new roads.
- Each line has two integers a and b : a new road is constructed between cities a and b .
- You may assume that every road will be constructed between two different cities.

Output: Print m lines: the required information after each day.

Sample Input	Sample Output	Explanation
5 3 1 2 1 3 4 5	4 2 3 3 2 3	{1,2} {3} {4} {5} {1,2,3} {4} {5} {1,2,3} {4,5}
7 10 1 4 5 6 4 7 1 7 3 5 1 2 2 7 3 6 2 5 5 4	6 2 5 2 4 3 4 3 3 3 2 4 2 4 2 4 1 7 1 7	{1,4} {2} {3} {5} {6} {7} {1,4} {2} {3} {5,6} {7} {1,4,7} {2} {3} {5,6} No change {1,4,7} {2} {3,5,6} {1,2,4,7} {3,5,6} No change No change No change {1,2,3,4,5,6,7} No change

Task 2: Check if a Graph is k-Regular

You are given an undirected graph with a set of edges. Your task is to determine whether the graph is regular. A graph is considered regular if all vertices have the same number of edges.

Input:

- The first line contains two integers V and E representing the total number of vertices and edges, respectively.
- The following E lines contain edge information (u,v) where each pair represents an edge between two vertices (with no loops or multiple edges).

Output:

- Print "Yes" if the graph is regular.
- Print "No" if the graph is not regular.

Sample Input	Sample Output	Explanation
4 4 1 2 2 3 3 4 4 1	Yes	Each vertex has exactly 2 edges: Vertex 1 has edges (1-2), (1-4) $\rightarrow$ degree = 2 Vertex 2 has edges (2-1), (2-3) $\rightarrow$ degree = 2 Vertex 3 has edges (3-2), (3-4) $\rightarrow$ degree = 2 Vertex 4 has edges (4-1), (4-3) $\rightarrow$ degree = 2 The graph is 2-regular.
5 6 1 2 2 3 3 4 4 5 5 1 2 4	No	Vertex 1 has edges (1-2), (1-5) $\rightarrow$ degree = 2 Vertex 2 has edges (2-1), (2-3), (2-4) $\rightarrow$ degree = 3 Vertex 3 has edges (3-2), (3-4) $\rightarrow$ degree = 2 Vertex 4 has edges (4-3), (4-5), (4-2) $\rightarrow$ degree = 3 Vertex 5 has edges (5-1), (5-4) $\rightarrow$ degree = 2 Not all vertices have the same degree, so the graph is not k-regular.
5 10 1 2 2 3 1 3 3 5 3 4 4 5 1 4 1 5 2 4 2 5	Yes	

Task 3: (10 points) Implement Breadth First Search (BFS) for an undirected, unweighted graph.

Given a graph $G = (V, E)$ and a source vertex 's':

- Implement the BFS algorithm to discover every vertex reachable from 's'.
- Compute the distance of every reachable vertex from s.
- Print the path of each reachable vertex from s.
- Print the list of edges that produce the BFS tree.

Each nodes has three attributes: distance, predecessor, and color.

Input:

The first line will contain V, E, and s representing the total number of vertices, edges, and the source vertex. Following E lines will contain the edge information.

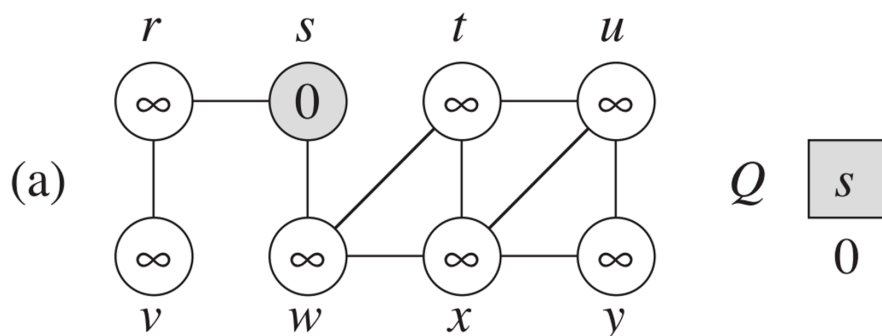
Output:

1. (1 points) After storing the Graph, first print the adjacency list
2. (2 points) Print the order in which nodes will be discovered in BFS
3. (2 points) Print the distance of each node from source
4. (3 points) Print the path to reach each vertex from the source
5. (2 points) Print the edges of the BFS tree

Sample Input	Sample Output
<pre> 8 10 2 2 6 2 1 6 3 6 7 1 5 3 4 3 7 7 8 7 4 4 8 </pre>	<pre> Adjacency list: 1: 2 5 2: 6 1 3: 6 4 7 4: 3 7 8 5: 1 6: 2 3 7 7: 6 3 8 4 8: 7 4 BFS order: 2 6 1 3 7 5 4 8 Distance from source: 1(1) 2(0) 3(2) 4(3) 5(2) 6(1) 7(2) 8(3) Paths from source: 1: 2->1 2: 2 3: 2->6->3 4: 2->6->3->4 5: 2->1->5 6: 2->6 7: 2->6->7 8: 2->6->7->8 Edges of BFS tree: (order may vary) 2 6 2 1 6 3 6 7 1 5 3 4 7 8 </pre>

Steps:

- Use pushback to insert new nodes in adjacency list. This should result into the given output list.
- Don't implement everything at once. First implement the BFS algorithm, then show the distance, then the path, and finally the tree-edge list. There are separate marks for each step.
- Following figure is taken from the lecture slide. We considered $\{r,s,t,u,v,w,x,y\}$ as $\{1,2,3,\dots,8\}$. Hence simulation should match with the slide.



- Use the following pseudocode for BFS algorithm

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 

```

Task 4: (10 points) Implement Depth First Search (DFS) for a directed unweighted graph.

Given a graph $G = (V, E)$:

- Implement the DFS algorithm to discover every vertex.
- Classify the type of each edge.

Each nodes has four attributes: discovery time, finishing time, predecessor, and color.

Input:

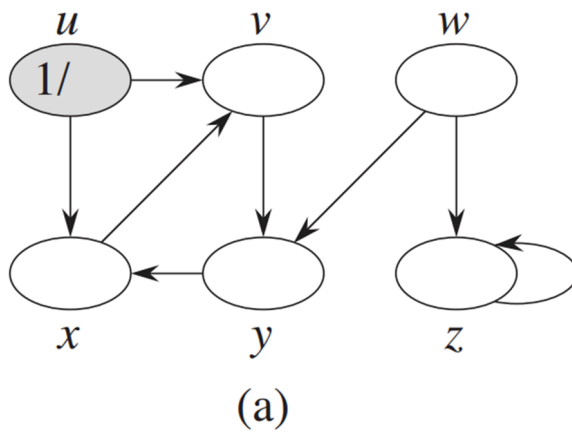
First line will contain V , and E representing the total number of vertices, and edges. Following E lines will contain the edge information.

Output:

1. (1 point) After storing the Graph, first print the adjacency list
2. (3 points) Print the order in which nodes will be discovered in DFS
3. (2 points) Print the discovery and finishing timestamps of each node
4. (4 points) Print class type of each edge

Sample Input	Sample Output
<pre> 6 8 1 2 1 4 2 5 5 4 4 2 3 5 3 6 6 6 </pre>	<pre> Adjacency list: 1: 2 4 2: 5 3: 5 6 4: 2 5: 4 6: 6 DFS order: 1 2 5 4 3 6 Timestamps of Vertex(discovery/finishing): 1(1/8) 2(2/7) 3(9/12) 4(4/5) 5(3/6) 6(10/11) Edge classification: 1 2: Tree Edge 1 4: Forward Edge 2 5: Tree Edge 5 4: Tree Edge 4 2: Back Edge 3 5: Cross Edge 3 6: Tree Edge 6 6: Back Edge </pre>

- The following figure is taken from the lecture slide. We considered $\{u,v,w,x,y,z\}$ as $\{1,2,3,\dots,6\}$. Hence, the simulation should match the slide.



- Use the following pseudocode for DFS:

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )

```

DFS-VISIT(G, u)

```

1   $time = time + 1$                 // white vertex  $u$  has just been discovered
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$           // explore edge  $(u, v)$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$               // blacken  $u$ ; it is finished
9   $time = time + 1$ 
10  $u.f = time$ 

```